

GlobalExceptionHandler Class – Tanglish Explanation

```
package com.inventory.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<
handleValidationErrors(MethodArgumentNotValidException ex) {
    Map errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(err ->
errors.put(err.getField(), err.getDefaultMessage()));
    return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
}
}
```

■ ****Class Name:**** `GlobalExceptionHandler` ■ **Purpose:** Spring Boot la global ah exception handle panna use pannum class idhu. ■ ****Annotation:** `@RestControllerAdvice` Idhu use panni, app la yedha controller la error vandhalum, idha call pannum. So centralized exception handling. ■■ ****Method:**** `handleValidationErrors()` – Parameter: `MethodArgumentNotValidException ex` – Spring form validation fail aana time la idhu trigger aagum. ■ ****Inside Method Explanation:**** 1■■ `Map errors = new HashMap<>();` → Error details store panna map create pannum. 2■■ `ex.getBindingResult().getFieldErrors()` → Validation fail aana field list edukkum. 3■■ `.forEach(err -> errors.put(err.getField(), err.getDefaultMessage()));` → Field name + error message map la add pannum. 4■■ `return new ResponseEntity<>(errors, HttpStatus.BAD\_REQUEST);` → Client ku 400 (Bad Request) oda map of errors return pannum. ■ ****Output Example:**** User name empty ah irundha -> `{ "name": "Name cannot be blank" }`

maari json return pannum. ■ Summary: – Centralized error handling. – Reduces code duplication. – Improves response clarity for clients.